

Projet IoT Master 1 IA

Sujet : *CuveGuard* — surveillance d'un réservoir d'eau avec remplissage automatique.

Date limite : vendredi 17 juillet 2026, 22h00. Aucun délai supplémentaire.

Rendu : un fichier .zip en message privé Slack. Nom du fichier :

`epreuve_cuve_GROUPE1.zip`

(adaptez le numéro de groupe : GROUPE2, GROUPE3, etc.).

Contenu du zip :

- projet Wokwi exporté (`diagram.json`, `sketch.ino`, `libraries.txt`);
- scripts Python (`requirements.txt` inclus);
- fichier de config **exemple** (tokens masqués);
- export JSON du dashboard ThingsBoard;
- **vidéo de démonstration** (`demo.mp4` ou `demo.webm`, 15 min max);
- `README.md`.

Ne pas inclure vos tokens ni mots de passe réels.

1. Problème

On surveille une cuve d'eau. Un HC-SR04 mesure la distance jusqu'à la surface; vous en déduisez un **niveau en pourcentage**. Un DHT22 mesure température et humidité. Une pompe de remplissage est simulée par un **relais + LED**. Une LED rouge ou un buzzer signale un **niveau bas**.

Le système doit tourner en **automatique** : un script Python lit la télémétrie sur ThingsBoard et commande la pompe par RPC. Le dashboard ThingsBoard permet aussi de piloter la pompe manuellement.

Comportement attendu

- Niveau < **30 %** : pompe en marche (mode auto).
- Niveau > **90 %** : pompe arrêtée (mode auto).
- Éviter les allumages/extinctions en rafale (hystérésis ou délai).
- Alerte locale si niveau < **15 %**.

2. Wokwi

Matériel **uniquement** depuis le catalogue Wokwi : ESP32, HC-SR04, DHT22, module relais, LED(s), résistances, buzzer si besoin.

Le firmware se connecte au Wi-Fi Wokwi et publie la télémétrie vers ThingsBoard en MQTT. Il doit traiter les **RPC** reçues depuis ThingsBoard (commande pompe, et mode manuel/auto si vous l'implémentez).

En simulation, la pompe ne remplit pas la cuve toute seule : pour faire varier le niveau, **ajustez la distance du HC-SR04 via le slider Wokwi** (clic sur le capteur pendant la simulation).

3. ThingsBoard

Plateforme : `eu.thingsboard.cloud`. Deux appareils au minimum :

- un pour l'ESP32 (firmware Wokwi) ;
- un pour l'agent Python.

Télémétrie ESP32 (clés imposées)

Clé	Type	Signification
<code>distanceCm</code>	nombre	Distance mesurée par le HC-SR04 (cm).
<code>waterLevelPct</code>	nombre	Niveau d'eau calculé, de 0 à 100.
<code>temperature</code>	nombre	Température ambiante (DHT22), en °C.
<code>humidity</code>	nombre	Humidité relative (DHT22), en %.
<code>pumpOn</code>	booléen	true si la pompe tourne, false sinon.
<code>manualMode</code>	booléen	true si la pompe est pilotée à la main (RPC dashboard), false en mode auto.
<code>alertLevel</code>	entier	0 = normal ($\geq 30\%$) ; 1 = attention (entre 15 et 30 %) ; 2 = critique ($< 15\%$).

Télémétrie agent Python (clés imposées)

Clé	Type	Signification
<code>agentDecision</code>	texte	Dernière décision : START (démarrer pompe), STOP (arrêter), ou HOLD (ne rien changer).
<code>observedLevelPct</code>	nombre	Niveau lu sur ThingsBoard au moment de la décision.
<code>pumpCommandSent</code>	booléen	true si une RPC a été envoyée à l'ESP32 lors de ce cycle.
<code>autoMode</code>	booléen	true si l'agent applique les seuils ; false s'il laisse le pilotage manuel.

RPC (commandes vers l'ESP32)

Le firmware doit répondre à au moins :

- `setPump` — paramètre `value` : **true** ou **false** ;
- `setManualMode` — paramètre `value` : **true** ou **false**.

En mode manuel, l'agent Python ne doit pas forcer la pompe.

Dashboard (éléments obligatoires)

Créer un dashboard ThingsBoard, l'exporter en JSON et le mettre dans le zip. Il doit contenir au minimum :

- **Jauge ou carte** : `waterLevelPct` (KPI principal) ;
- **Graphique temporel** : `temperature` et `humidity` ;
- **Graphique temporel** : `waterLevelPct` (ou `distanceCm`) ;
- **Indicateur** : `pumpOn` (état pompe) ;
- **Indicateur** : `alertLevel` (normal / attention / critique) ;
- **Widgets agent** : `agentDecision` et `observedLevelPct` ;
- **Commande RPC** : un **switch** ou des **boutons** sur le dashboard pour allumer/éteindre la pompe (`setPump`) et basculer manuel/auto (`setManualMode`).

4. Python

Un ou plusieurs scripts qui :

- lisent `waterLevelPct` (et `manualMode`) depuis ThingsBoard ;
- appliquent les seuils 30 % / 90 % ;
- envoient la RPC `setPump` si besoin ;
- publient la télémétrie agent (clés ci-dessus) ;
- tournent en boucle sans intervention une fois lancés.

Organisation des fichiers : **libre**, tant que le zip reste clair.

5. Vidéo de démonstration

Fichier vidéo (**15 minutes maximum**) inclu dans le zip. Montrer clairement :

1. le montage Wokwi en fonctionnement (Serial Monitor ou comportement visible) ;
2. ThingsBoard : télémétrie qui arrive, graphes qui se mettent à jour ;
3. au moins **deux scénarios testés** devant la caméra, par exemple :
 - baisser le niveau avec le **slider HC-SR04** → pompe qui démarre en auto ;
 - remonter le niveau (slider) au-dessus de 90 % → pompe qui s'arrête ;
 - utiliser le switch/bouton RPC du dashboard pour forcer la pompe en manuel.
4. l'agent Python lancé et ses valeurs visibles sur le dashboard.

6. README

Le `README.md` sert à donner des **détails supplémentaires si nécessaire** : prérequis, lancement Wokwi, configuration ThingsBoard, exécution du script Python, import du dashboard. Pas besoin de répéter tout l'énoncé ; complétez seulement ce qui n'est pas évident dans votre projet.

Travail en groupe, un rendu par groupe. Montage sur Wokwi uniquement ; pas de matériel physique exigé.